

Unit - 2

Constructor:- It is a special member function of a class that automatically called when object of the class is created.

Key characteristic of constructor-

- (i) Same name as a class
- (ii) There is no return type. ~~It means~~
- (iii) It can be overloaded.
- (iv) Constructor are basically called in public section but they may be accessed from private & protected section.

Types of constructor -

- (1) Default constructor
- (2) Parameter / Parameterized constructor.
- (3) Copy Constructor.

(1) Default constructor:-

```
#include <iostream.h>
#include <conio.h>
class add
{
    public :
        add()
        {
            cout << "Default Constructor" << endl;
        }
};
```

```
void main()
```

```
{
```

```
    class add a1; [called automatically]
```

```
    clrscr();
```

```
    getch();
```

```
}
```

For outside the class.

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class add
```

```
{
```

```
    public :
```

```
        add()
```

```
};
```

```
void add :: add()
```

```
{
```

```
    cout << "Default constructor" << endl;
```

```
}
```

```
void main()
```

```
{
```

```
    class add a1;
```

```
    clrscr();
```

```
    getch();
```

```
}
```

(2) Parameterised constructor :-

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class add
```

```
{
```

```
    int a, b;
```

```
public:
```

```
    void add()
```

```
    { cout << "Constructor called" << endl;
```

```
    void add(int x, int y)
```

```
    {
```

```
        a = x;
```

```
        b = y;
```

```
        cout << "a =" << a << endl;
```

```
        cout << "b =" << b << endl;
```

```
    }
```

```
};
```

```
void main()
```

```
{
```

```
    class add a1;
```

```
    class add a2(5, 7);
```

```
    getch();
```

```
}
```

xx

[Constructor over-loading]

(3) Copy Constructor :-

```
class A
{
public:
    int x;
};
```

```
int main ( )
{
    class A obj1;
    obj1.x = 10;
    cout << "obj1.x = " <<
        obj1.x << endl;
```

```
    class A obj2 (obj1);
    cout << "obj2.x = " << obj1.x
        << endl;
```

```
    return 0;
```

```
}
```

output

obj1.x = 10

obj2.x = 10

A copy constructor is a type of constructor that create an object using another object of the same class.

public is used to access the value of constructor easily. private declar doesn't support

Note

~A() => destructor.

used to destroy memory of previous object from constructor.

(pointer in constructor)

* User define Copy constructor :

```
class A
{
    public :
    int x ;
    A ()
    {
    } ;
    A (A &obj)
    {
        x = obj.x ;
        cout << "user defined copy constructor "
              << endl ;
    } ;
};

int main ()
{
    class A obj1 ;
    obj1.x = 10 ;
    cout << "obj1.x = " << obj1.x << endl ;
    class A obj2 (obj1) ;
    cout << "obj2.x = " << obj2.x << endl ;
    return 0 ;
}
```

output

obj1.x = 10

user defined copy
constructor

obj2.x = 10

Dynamic memory allocation:

The new operator in C++ is used to allocate the memory in dynamic way or we can say that 'new' allocate memory in more flexible way.

'delete' is used for deallocation.

```
[int *ptr = new int;]
```

↳ allocate Integer memory
2 or 4 byte.

```
*ptr = 35;  
cout << *ptr;  
delete ptr;
```

```
#include <iostream>  
using namespace std;  
int main()  
{  
    int *p = new int;  
    *p = 10;  
    cout << " *p = " << *p << endl;  
    delete p;  
    p = Null ptr;  
    return 0;  
}
```

* pointer dangling (p = Null ptr;)

Q. Imagine you run a hotel & since you don't know how many guests will arrive. You ask user a number to create many rooms.

Solⁿ

```
#include <iostream.h>
#include <string.h>
int main()
{
    int numguest;
    cout << "How many guest" << endl;
    cin >> numguest;
    if (numguest <= 0)
    {
        cout << "should be atleast one"
        return 0;
    }
}
```

```
String* guests = new string [numguests];
for (int i = 0; i < numguest; i++)
{
    cout << "Enter guest name" << (i+1);
    getline (cin, guest[i]);
}
cout << "guest checked in\n";
```

```

for (int i = 0; i < guest; i++)
{
    cout << guest[i] << endl;
}
delete [] guest;
return 0;
}

```

output

How many guest
3

Enter guest name 1
Ranchan Prajapati

Enter guest name 2
Komal Gurjar

Enter guest name 3
Khushi Agarwal

getline
accept whitespace
full line

✶ Inline Function :- The inline keyword suggest the compiler that the actual code is replaced with function code. Basically inline function is called overhead function.

perform inline function in some circumstance

- i) Function contain loop
- ii) Function contain static variable.
- iii) Function is recursive /iv, Switch & goto statement


```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class A
```

```
{  
public:
```

```
    inline int square(a);
```

```
};
```

```
class A::int square(a)
```

```
{
```

```
    return a * a;
```

```
}
```

```
int main ()
```

```
{
```

```
    cout << "Square is =" << square(3);
```

```
    return 0;
```

```
}
```

(Reduce time complexity)